YouTube Transcript

[00:13] [music]

[00:20] Thanks everybody for uh for joining. I'm

[00:23] here to talk to you today about DSPI.

[00:26] Um, and feel free to jump in with

[00:28] questions or anything throughout the

[00:30] talk. It's, you know, I don't sp I don't

[00:32] plan on spending the full hour and a

[00:34] half or so. I know it's the last session

[00:35] of the day. So, um, keep it casual. Feel

[00:38] free to jump in. I'll start with a

[00:40] little bit of background. Don't want to

[00:41] go through too many slides. I'm

[00:44] technically a consultant, so I have to

[00:45] do some slides, but we will dive into

[00:47] the code for the the latter half. And

[00:50] there's a GitHub repo that you can

[00:51] download to to follow along and play

[00:53] around with it on your own.

[00:56] Um, so how many people here have heard

[01:00] of DSPI?

[01:02] Almost everyone. That's awesome. How

[01:04] many people have actually used it kind

[01:05] of day-to-day in production or anything

[01:08] like that? Three. Okay, good. So

[01:10] hopefully we can convert some more of

[01:12] you today. Um, so high level DSPI, this

[01:16] is straight from the website. Um, it's a

[01:18] declarative framework for how you can

[01:20] build modular software. And most

[01:23] important for someone like myself, I'm

[01:25] not necessarily an engineer that is

[01:27] writing code all day, every day. As I

[01:30] mentioned before, I'm a more of a

[01:31] technical consultant. So, I run across a

[01:33] variety of different problems. Could be

[01:36] um an investigation for a law firm. It

[01:38] could be helping a company understand

[01:40] how to improve their processes, how to

[01:42] deploy AI internally. Maybe we need to

[01:44] look through 10 10,000 contracts to

[01:46] identify a particular clause um or or

[01:49] paragraph. And so DSPI has been a really

[01:52] nice way for me personally and my team

[01:53] to iterate really really quickly on

[01:55] building these applications.

[01:59] Most importantly building programs. It's

[02:01] not um it's not kind of iterating with

[02:04] prompts and tweaking things back and

[02:06] forth. It is building a a proper Python

[02:09] program and and DSP is a really good way

[02:11] for you to do that.

[02:13] So I mentioned before there's a repo

[02:15] online if you want to download it now

[02:17] and kind of just get everything set up.

[02:19] I'll put this on the screen later on.

[02:21] Um, but if you want to go here, just

[02:23] kind of download some of the code. It uh

[02:26] it's been put together over the past

[02:27] couple days. So, it's not going to be

[02:29] perfect production level code. It's much

[02:31] more of utilities and little things here

[02:33] and there to just come and kind of

[02:34] demonstrate the usefulness, demonstrate

[02:36] the point of of what we're talking about

[02:38] today in that and we'll walk through all

[02:41] of these these different use cases. So

[02:44] um sentiment classifier going through a

[02:46] PDF some multimodal work uh a very very

[02:49] simple web research agent detecting

[02:52] boundaries of a PDF document you'll see

[02:54] how to summarize basically arbitrary

[02:56] length text and then go into an

[02:59] optimizer uh with Jeepo

[03:02] but before we do that just again kind of

[03:04] level set the biggest thing for me

[03:06] personally DSP is a really nice way to

[03:09] decompose your logic into a program that

[03:12] treats LLMs as a first class citizen. So

[03:15] at the end of the day, you're

[03:16] fundamentally just calling a function

[03:19] that under the hood just happens to be

[03:21] an LLM and DSPI gives you a really nice

[03:23] intuitive easy way to do that with some

[03:27] guarantees about the input and output

[03:29] types. So of course there are structured

[03:31] outputs, of course there are other ways

[03:32] to do this, Pyantic [snorts]

[03:34] and others. Um, but DSPI has a set of

[03:38] primitives that when you put it all

[03:40] together allows you to build a cohesive

[03:43] modular piece of software that you then

[03:46] happen to be able to optimize. We'll get

[03:48] into that uh in a minute.

[03:51] So, just a few reasons of why I'm such

[03:53] an advocate. It sit at it sits at this

[03:55] really nice level of abstraction. So,

[03:57] it's I I would say it doesn't get in

[04:00] your way as much as a lang chain. And

[04:02] that's not a knock-on lang chain. It's

[04:04] just a different kind of paradigm in the

[04:06] way that DSPI is is structured. Um, and

[04:09] allows you to focus on things that

[04:11] actually matter. So you're not writing

[04:14] choices zero messages content. You're

[04:16] not you're not doing string parser.

[04:18] You're not doing a bunch of stuff under

[04:19] the hood. You're just declaring your

[04:21] intent of how you want the program to

[04:22] operate, what you want your inputs and

[04:24] outputs to be.

[04:27] Because of this, it allows you to create

[04:28] computer programs. As I mentioned

[04:30] before, not just tweaking strings and

[04:32] sending them back and forth. You are

[04:34] building a program first. It just

[04:36] happens to also use LLMs. And really the

[04:39] the most kind of important part of this

[04:41] is that and Omar the KB the uh the

[04:44] founder of this or the the original

[04:46] developer of it had this really good

[04:48] podcast with A16Z. I think it came out

[04:50] just like two or three days ago. But it

[04:53] he put it a really nice way. He said

[04:54] it's a it's built with a systems mindset

[04:56] and it's really about how you're

[04:58] encoding or expressing your intent of

[05:01] what you want to do most importantly in

[05:03] a way that's transferable. So the the

[05:06] design of your system, I would imagine,

[05:08] or your program isn't going to move

[05:10] necessarily as quickly as maybe the

[05:13] model capabilities are under the hood.

[05:15] when we see new releases almost every

[05:17] single day, different capabilities,

[05:19] better models and so DSPI allows you to

[05:22] structure it in a way that retains the

[05:26] control flow uh retains the intent of

[05:29] your system, your program um while

[05:31] allowing you to bounce from model to

[05:33] model to the extent that you want to or

[05:35] need to.

[05:36] Convenience comes for free. There's no

[05:38] parsing, JSON, things like that. It

[05:40] again, it sits at a nice level of

[05:41] abstraction where you can still

[05:43] understand what's going on under the

[05:45] hood. If you want to, you can go in and

[05:47] tweak things, but it allows you to to

[05:49] kind of focus on just what you want to

[05:50] do while retaining the level of

[05:52] precision that you that I think most of

[05:54] us would like to have in and kind of

[05:56] building your programs. Um, [snorts]

[05:58] as mentioned, it's robust to kind of

[06:00] model and paradigm shifts. So, you can

[06:02] again keep the logic of your program. Um

[06:04] but it but keep that those LLMs infused

[06:07] in uh basically in line. Now that being

[06:11] said, you know, there are absolutely

[06:12] other great libraries out there.

[06:14] Pedantic AI, Langchain, there's many

[06:16] many others that allow you to do similar

[06:18] things. Agno is another one. Um this is

[06:21] just one perspective and um it may not

[06:24] be perfect for your use case. For me, it

[06:26] took me a little bit to kind of gro how

[06:30] DSPI works and you'll see why that is in

[06:32] a minute. Um, so I would just recommend

[06:35] kind of have an have an open mind, play

[06:36] with it. Um, run the code, tweak the

[06:39] code, do whatever you need to do. Um,

[06:41] and just see how it might work, might

[06:43] work for you. And really, this talk is

[06:46] more about ways that I found it useful.

[06:48] It's not a dissertation on the ins and

[06:50] outs of every nook and cranny of DSPI.

[06:53] It's more of, you know, I've run into

[06:55] these problems myself now. I naturally

[06:57] run to DSPI to solve them. And this is

[07:00] kind of why. And the hope is that you

[07:01] can extrapolate some of this to your own

[07:04] use cases. So we we'll go through

[07:06] everything uh fairly quickly here, but

[07:09] the core concepts of DSPI really comes

[07:12] down to arguably five or these six that

[07:15] you see on the screen here. So we'll go

[07:17] into each of these in more detail, but

[07:19] high level signatures

[07:22] specify what you want the L what

[07:24] basically what you want your function

[07:26] call to do. This is when you specify

[07:28] your inputs, your outputs. Inputs and

[07:31] outputs can both be typed. Um, and you

[07:34] defer the rest of the basically the how

[07:37] the implementation of it to the LLM. And

[07:39] we'll see how we how that all kind of

[07:41] comes together uh in a minute. Modules

[07:44] themselves are ways to logically

[07:47] structure your program. They're based

[07:49] off of signatures. So, a module can have

[07:51] one or more signatures embedded within

[07:53] it in addition to uh additional logic.

[07:56] and it's based off of um pietorrch and

[08:00] some of the in terms of like the

[08:01] methodology for how it's structured and

[08:03] you'll you'll see how that uh comes to

[08:05] be in a minute. Tools we're all familiar

[08:08] with tools MCP and others and really

[08:11] tools fundamentally as DSPI looks at

[08:14] them are just Python functions. So it's

[08:16] just a way for you to very easily expose

[08:19] Python functions to the LLM within the

[08:22] DSP kind of ecosystem if you will. um

[08:26] adapters

[08:29] live in between your signature and the

[08:33] LLM call itself. I mean, as we all know,

[08:35] prompts are ultimately just strings of

[08:37] text that are sent to the LLM.

[08:40] Signatures are a way for you to express

[08:42] your intent at a at a higher level. And

[08:44] so, adapters are the things that sit in

[08:47] between those two. So, it's how you

[08:49] translate your inputs and outputs into a

[08:52] format basically explodes out from your

[08:55] initial signature into a format that is

[08:57] ultimately the prompt that is sent to

[08:59] the LLM. And so, you know, there's some

[09:03] debate or some research on if certain

[09:05] models perform better with XML as an

[09:08] example or BAML or JSON or others. And

[09:12] so adapters give you a nice easy

[09:14] abstraction to to basically mix and

[09:16] match those at at will as you want.

[09:20] Optimizers um are

[09:24] the most interesting and for whatever

[09:26] reason the most controversial part of

[09:27] DSP. That's kind of the first thing that

[09:29] people think of or at least when they

[09:31] hear of DSP they think optimizers. We'll

[09:34] see a quote in a minute. It's not

[09:35] optimizers first. It is just a nice

[09:38] added benefit and a nice capability that

[09:41] DSPI offers in addition to the ability

[09:43] to structure your program with the

[09:45] signatures and modules and everything

[09:47] else. Um, and metrics are used in tandem

[09:51] with optimizers that that basically

[09:52] defines how you measure success in your

[09:56] in uh your DSPI program. So the

[09:58] optimizers use the metrics to determine

[10:01] if it's finding the right path if you

[10:03] will.

[10:05] So signature as I mentioned before it's

[10:07] how you express your intent your

[10:08] declarative intent can be super simple

[10:11] strings and this is the weirdest part

[10:12] for me initially but is one of the most

[10:15] powerful parts uh of it now or it can be

[10:18] more complicated class-based classbased

[10:21] objects if you've used pyantic it that's

[10:24] basically what what it runs on under the

[10:25] hood.

[10:27] So this is an example of one of the

[10:29] class-based signatures. Again, it it's

[10:32] basically just a pyantic object.

[10:35] What's super interesting about this is

[10:37] that

[10:39] the

[10:40] the names of the fields themselves act

[10:43] almost as like mini prompts. It's part

[10:45] of the prompt itself. And you'll see how

[10:47] this comes to life in a minute. But

[10:49] what's ultimately passed to the model

[10:52] from something like this is it will say

[10:54] okay your inputs are going to be a

[10:57] parameter called text and it's based off

[10:59] of the name of the that particular

[11:03] parameter in this class. And so these

[11:05] things are actually passed through. And

[11:06] so it's it's very important uh to be

[11:10] able to name your parameters in a way

[11:13] that is intuitive for the model to be

[11:14] able to pick it up. Um, and you can add

[11:17] some additional context or what have you

[11:19] in the description field here. So most

[11:22] of this, if not all of this, yes, it is

[11:24] proper, you know, typed Python code, but

[11:27] it's also it also serves almost as a

[11:30] prompt ultimately that feeds into the

[11:31] model. Um, and that's basically

[11:33] translated through the use of adapters.

[11:36] Um, and so just to highlight here like

[11:38] these, it's the ones that are a little

[11:40] bit darker and bold, you know, those are

[11:42] the things that are effectively part of

[11:44] the prompt. uh that's been sent in and

[11:47] you'll see kind of how DSPI works with

[11:49] all this and formats it in a way that

[11:51] again allow you to just worry about what

[11:53] you want. Worry about constructing your

[11:55] signature instead of figuring out how

[11:58] best to word something in the prompt. Go

[12:00] >> ahead.

[12:05] I have a really good prompt.

[12:07] >> Sure. Then I don't want this thing.

[12:17] >> That's exactly right.

[12:25] >> Sure.

[12:27] >> So the the question for folks online is

[12:29] what if I already have a great prompt?

[12:31] I've done all this work. I'm a I'm a

[12:33] amazing prompt engineer. I don't want my

[12:35] job to go away or whatever. Um, yes. So,

[12:39] you can absolutely start with a custom

[12:41] prompt or something that you have

[12:43] demonstrated works really well. And

[12:45] you're exactly right that's that can be

[12:47] done in the dock string itself. There's

[12:49] there's some other methods in order uh

[12:51] for you to inject basically system

[12:53] instructions or add additional things at

[12:55] certain parts of the ultimate prompt and

[12:57] or of course you can just inject it in

[13:00] the in the final string anyway. I mean

[13:02] it's just you know a string that is

[13:04] constructed by VSPI. So um absolutely

[13:08] this doesn't necessarily prevent you it

[13:10] does does not prevent you from adding in

[13:12] some super prompt that you already have.

[13:14] Absolutely. Um and to your point it is

[13:18] it can serve as a nice starting point

[13:20] from which to build the rest of the

[13:21] system.

[13:23] Here's a shorthand version of the same

[13:25] exact thing which to me the first time I

[13:28] saw this so this was like baffling to

[13:29] me. Um, but it it that's exactly how it

[13:32] works is that you're basically again

[13:34] kind of deferring the implementation or

[13:37] the logic or what have you to DSPI and

[13:39] the model to basically figure out what

[13:41] you want to do. So in this case, if I

[13:44] want a super super simple text uh

[13:46] sentiment classifier, this is basically

[13:48] all you need. You're just saying, okay,

[13:50] I'm going to give you text as an input.

[13:52] I want the sentiment as an integer as

[13:54] the output. Now you probably want to

[13:55] specify some additional instructions to

[13:57] say okay your sentiment you know a lower

[14:00] number means negative you know a higher

[14:03] number is more positive sentiment etc.

[14:05] But it just gives you a nice kind of

[14:07] easy way to to kind of scaffold these

[14:09] things out in a way that you don't have

[14:11] to worry about like you know creating

[14:13] this whole prompt from hand. It's like

[14:15] okay I just want to see how this works

[14:17] and then if it works then I can add the

[14:18] additional instructions then I can

[14:20] create a module out of it or you know

[14:22] whatever it might be. It's these

[14:24] shorthand

[14:26] or it is this shorthand that makes

[14:28] experimentation and iteration incredibly

[14:30] quick.

[14:32] So modules it's that base abstraction

[14:34] layer for DSPI programs. There are a

[14:36] bunch of modules that are built in and

[14:39] these are a collection of kind of

[14:42] prompting techniques if you will and you

[14:44] can always create your own module. So to

[14:46] the question before, if you have

[14:47] something that you know works really

[14:49] well, sure yeah, put it in the module.

[14:51] That's now the kind of the base

[14:53] assumption, the base module that others

[14:55] can build off of. And all of DSPI is

[14:59] meant to be composable, optimizable, and

[15:03] when you deconstruct your business logic

[15:05] or whatever you're trying to achieve by

[15:07] using these different primitives, it all

[15:09] it's intended to kind of fit together

[15:11] and flow together. Um, and we'll get to

[15:13] optimizers in a minute, but at least for

[15:15] me and my team's experience, just being

[15:18] able to logically separate the different

[15:20] components of a program, but basically

[15:23] inlining uh LLM calls has been

[15:25] incredibly powerful for us. And it's

[15:27] just an added benefit that at the end of

[15:29] the day, because we're just kind of in

[15:32] the DSPI paradigm, we happen to also be

[15:34] able to optimize it at the end of the

[15:36] day. Uh, so it comes with a bunch of

[15:38] standard ones built in. I I don't use

[15:41] some of these bottom ones as much,

[15:43] although it's they're super interesting.

[15:45] Um the base one at the top there is just

[15:48] DSpi.predict.

[15:50] That's literally just, you know, an LM

[15:52] call. That's just uh a vanilla call.

[15:55] chain of thought uh probably isn't isn't

[15:58] as relevant anymore these days because

[16:00] models have kind of ironed those out but

[16:03] um it is a good example of the types of

[16:08] um kind of prompting techniques that can

[16:10] be built into some of these modules um

[16:12] and basically all this does is add um

[16:15] some some of the uh strings from

[16:17] literature to say okay let's think step

[16:19] by step or whatever that might be same

[16:21] thing for react and codeact react is

[16:24] basically the way that you expose the

[16:26] tools to the model. So, it's wrapping

[16:28] and doing some things under the hood

[16:30] with um basically taking your signatures

[16:33] and uh it's injecting the Python

[16:36] functions that you've given it as tools

[16:38] and basically React is how you do tool

[16:40] calling in DSP.

[16:43] Program with thought is uh is pretty

[16:45] cool. It kind of forces the model to

[16:48] think in code and then we'll return the

[16:50] result. Um, and you can give it a, it

[16:54] comes with a Python interpreter built

[16:55] in, but you can give it some custom one,

[16:57] some type of custom harness if you

[16:59] wanted to. Um, I haven't played with

[17:01] that one too too much, but it is super

[17:03] interesting. If you have like a highly

[17:05] technical problem or workflow or

[17:07] something like that where you want the

[17:08] model to inject reasoning in code at

[17:11] certain parts of your pipeline, that's

[17:13] that's kind of an really easy way to do

[17:14] it. And then some of these other ones

[17:16] are basically just different

[17:17] methodologies for comparing outputs or

[17:20] running things in parallel.

[17:23] So here's what one looks like. Again,

[17:25] it's it's fairly simple. It's, you know,

[17:27] it is a Python class at the end of the

[17:29] day. Um, and so you do some initial

[17:32] initialization up top. In this case,

[17:35] you're seeing the uh

[17:38] uh the shorthand signature up there. So,

[17:41] I'm this module uh just to give you some

[17:44] context is an excerpt from um one of the

[17:47] the Python um files that's in the repo

[17:50] is basically taking in a bunch of time

[17:53] entries and making sure that they adhere

[17:56] to certain standards, making sure that

[17:58] things are capitalized properly or that

[18:00] there are periods at the end of the

[18:01] sentences or whatever it might be.

[18:03] that's from a real client use case where

[18:05] they had hundreds of thousands of time

[18:07] entries and they needed to make sure

[18:08] that they all adhere to the same format.

[18:11] This was one way to to kind of do that

[18:12] very elegantly, at least in my opinion,

[18:15] was taking up top you can define the the

[18:20] signature. It's adding the some

[18:22] additional instructions that were

[18:23] defined elsewhere and then saying for

[18:25] this module the the change tense um call

[18:29] is going to be just a vanilla predict

[18:31] call. And then when you actually call

[18:33] the module, you enter into the forward

[18:35] function which you can inter basically

[18:38] intersperse the LLM call which would be

[18:40] the first one and then do some kind of

[18:42] hard-coded business logic beneath it.

[18:47] Uh tools as I mentioned before these are

[18:49] just vanilla kind of Python functions.

[18:52] It's the DSP's tool interface. So under

[18:55] the hood, DSPI uses light LLM. And so

[18:58] there needs to be some kind of coupling

[19:00] between the two, but fundamentally um

[19:04] any type of tool that would that you

[19:06] would use elsewhere, you can also use in

[19:08] in DSPI. And this is probably obvious to

[19:11] most of you, but here's just an example.

[19:13] You have two functions, get weather,

[19:15] search web. You include that with a

[19:18] signature. So in this case, I'm saying

[19:20] the signature is I'm going to give you a

[19:22] question. please give me an answer. I'm

[19:24] not even specifying the types. It's just

[19:26] going to infer what that means. Uh I'm

[19:29] giving it the get weather and the search

[19:31] web tools and I'm saying, okay, do your

[19:33] thing, but only go five rounds just so

[19:36] it doesn't spin off and do something

[19:37] crazy. And then a call here is literally

[19:40] just calling the React agent that I

[19:42] created above with the question, what's

[19:44] the weather like in Tokyo? We'll see an

[19:46] example of this in the code session, but

[19:48] basically what this would do is give the

[19:51] model the prompt, the tools, and let it

[19:54] do its thing.

[19:57] So adapters, before I cover this a

[19:58] little bit, they're basically prompt

[20:00] formatterers, if you will. So the

[20:04] description from the docs probably says

[20:05] it best. It's you know it takes your

[20:07] signature the inputs other attributes

[20:09] and it converts them into some type of

[20:11] message format that you have specified

[20:14] or that the adapter has specified and so

[20:17] as an example the JSON adapter taking

[20:20] say a pyantic object that we defined

[20:22] before this is the actual prompt that's

[20:24] sent into the LLM and so you can see the

[20:27] input fields so this would have been

[20:29] defined as okay clinical note type

[20:32] string patient info as a patient details

[20:35] object object which which would have

[20:37] been defined elsewhere and then this is

[20:39] the definition of the patient info. It's

[20:42] basically a JSON dump of that pantic

[20:44] object. Go ahead.

[20:46] >> So this idea there's like a base adapter

[20:48] default that's good for most cases and

[20:50] this is if you want to tweak that to do

[20:51] something more specific.

[20:52] >> That's right.

[20:53] >> Yeah. The question was if if there's a

[20:55] base adapter and would this be an

[20:57] example of where you want to do

[20:58] something specific? Answer is yes. So um

[21:02] it's a guy pashant who is um I have his

[21:05] Twitter at the end of this presentation

[21:06] but he's been great. [clears throat] He

[21:08] did some testing comparing the JSON

[21:10] adapter with the BAML adapter. Um and

[21:12] you can see just intuitively even even

[21:14] for us humans the way that this is

[21:16] formatted is a little bit more

[21:18] intuitive. It's probably more token

[21:20] efficient too just considering like if

[21:22] you look at the messy JSON that's here

[21:24] versus the I guess slightly better

[21:27] formatted BAML that's here. um can

[21:29] actually improve performance by you know

[21:32] five to 10 percent depending on your use

[21:34] case. So it's a good example of how you

[21:36] can format things differently. The the

[21:38] rest of the program wouldn't have

[21:39] changed at all. You just specify the

[21:41] BAML adapter and it totally changes how

[21:44] the information is presented under the

[21:46] hood to the LLM

[21:50] multimodality. I mean this obviously is

[21:52] more at the model level but DSPI

[21:54] supports multiple modalities by default.

[21:56] So images, audio, some others. Um, and

[21:59] the same type of thing, you kind of just

[22:01] feed it in as part of your signature and

[22:03] then you can get some very nice clean

[22:05] output. This allows you to work with

[22:07] them very, very, very easily, very

[22:08] quickly. And for those uh, eagle-eyed

[22:12] participants, you can see the first uh,

[22:15] lineup there is attachments. It's

[22:17] probably a lesserk known library.

[22:19] Another guy on Twitter is awesome. Uh,

[22:21] Maxim, I think it is. uh he created this

[22:24] library that just is basically a

[22:26] catch-all for working with different

[22:28] types of files and converting them into

[22:30] a format that's super easy to use with

[22:32] LLMs. Um he's a big DSPI fan as well. So

[22:36] he made basically an adapter that's

[22:38] specific to this. But that's all it

[22:40] takes to pull in images, PDFs, whatever

[22:44] it might be. You'll see some examples of

[22:46] that and it just makes at least has made

[22:48] my life super super easy.

[22:52] Here's another example of the same sort

[22:53] of thing. So this is a PDF of a form 4

[22:57] form. So, you know, public SEC form from

[23:00] Nvidia.

[23:02] Um, up top I'm just giving it the link.

[23:04] I'm saying, okay, attachments, do your

[23:07] thing. Pull it down, create images,

[23:09] whatever you're going to do. I don't

[23:10] need to worry about it. I don't care

[23:11] about it. This is super simple rag, but

[23:13] basically, okay, I want to do rag over

[23:16] this document. I'm going to give you a

[23:18] question. I'm going to give you the

[23:19] document and I want the answer. Um, and

[23:22] you can see how simple that is.

[23:24] Literally just feeding in the document.

[23:26] How many shares were sold? Interestingly

[23:29] here, I'm not sure if it's super easy to

[23:31] see, but you actually have two

[23:32] transactions

[23:33] here. So, it's going to have to do some

[23:35] math likely under the hood. And you can

[23:38] see here the thinking and the the

[23:41] ultimate answer. Go ahead.

[23:42] >> Is it on the rag step? Is it creating a

[23:45] vector store of some kind or creating

[23:47] embeddings and searching over those? Is

[23:48] there a bunch going on in the background

[23:50] there or what?

[23:51] >> This is poor man's rack. I should have

[23:52] clarified. This is this is literally

[23:55] just pulling in the document images and

[23:59] I think attachments will do some basic

[24:01] OCR under the hood. Um, but it doesn't

[24:03] do anything other than that. That's it.

[24:05] All we're feeding in here, the the

[24:07] actual document object that's being fed

[24:09] in, yeah, is literally just the text

[24:11] that's been OCRD. the images, the model

[24:14] does the rest.

[24:17] [sighs] All right, so optimizers uh

[24:19] let's see how we're doing. Okay. Um

[24:22] optimizers are super powerful, super

[24:24] interesting concept. It's been some

[24:26] research um that argues I think that

[24:30] it's just as performant if not in cert

[24:33] in certain situations more performant

[24:35] than fine-tuning would be for certain

[24:37] models for certain situations. there's

[24:39] all this research about in context

[24:40] learning and such. And so whether you

[24:43] want to go fine-tune and do all of that,

[24:46] nothing stops you. But I would recommend

[24:48] at least trying this first to see how

[24:50] far you can get without having to set up

[24:52] a bunch of infrastructure and, you know,

[24:54] go through all of that. See how the

[24:55] optimizers work. Um, but fundamentally

[24:59] what it allows you to do is DSPI gives

[25:01] you the primitives that you need and the

[25:03] organization you need to be able to

[25:06] measure and then quantitatively improve

[25:09] that performance. And I mentioned

[25:11] transferability before. This the

[25:14] transferability is enabled arguably

[25:18] through the use of optimizers because if

[25:20] you can get okay I want to I have the

[25:22] classification task works really well

[25:24] with 41 but maybe it's a little bit

[25:26] costly because I have to run it a

[25:27] million times a day. Can I try it with

[25:31] 41 nano? Okay, maybe it's at 70%

[25:34] whatever it might be. But I run the

[25:36] optimizer on 41 nano and I can get the

[25:39] performance back up to maybe 87%. maybe

[25:42] that's okay for my use case, but I've

[25:44] now just dropped my cross my cost

[25:45] profile by multiple orders of magnitude.

[25:48] And it's the optimizer that allows you

[25:50] to do that type of model and kind of use

[25:52] case transferability, if you will. But

[25:55] really all it does at at the end of the

[25:57] day under the hood is iteratively prompt

[26:00] uh iteratively optimize or tweak that

[26:03] prompt, that string under the hood. And

[26:05] because you've constructed your program

[26:07] using the different modules, DSPI kind

[26:09] of handles all of that for you under the

[26:11] hood. So if you compose a program with

[26:13] multiple modules and you're optimizing

[26:16] against all that, it it by itself DSPI

[26:19] will optimize the various components in

[26:21] order to improve the input and output

[26:24] performance.

[26:27] And we'll we'll take it from the man

[26:29] himself, Omar. You know, ESPI is not an

[26:32] optimizer. I've said this multiple

[26:34] times. it's it's just a set of

[26:35] programming abstractions or a way to

[26:37] program. You just happen to be able to

[26:39] optimize it. Um so again, the value that

[26:43] I've gotten and my team has gotten is

[26:45] mostly because of the programming

[26:46] abstractions. It's just this incredible

[26:48] added benefit that you are also able to

[26:51] to should you choose to to optimize it

[26:54] afterwards.

[26:55] And I was listening to this to Dwaresh

[26:57] and and uh Carpathy the other day and

[27:01] this kind of I was like prepping for

[27:04] this talk and this like hit home

[27:05] perfectly. I was thinking about the

[27:06] optimizers and someone smarter than me

[27:09] can can ple you know please correct me

[27:11] but I think this makes sense because he

[27:16] he was basically talking about using LLM

[27:18] as a judge can be a bad thing because

[27:21] the model being judged can find

[27:24] adversarial examples and degrade the

[27:27] performance or basically um create a

[27:30] situation where the judge is not uh not

[27:33] scoring something properly. um because

[27:35] he's saying that the model will find

[27:37] these little cracks. It'll find these

[27:38] little spirious things in the nooks and

[27:40] crannies of the giant model and find a

[27:42] way to cheat it. Basically saying that

[27:44] LM as a judge can only go so far until

[27:47] the other model uh finds those

[27:49] adversarial examples. If you kind of

[27:51] invert that and flip that on its head,

[27:53] it's this property that the optimizers

[27:55] for DSpir are taking advantage of to

[27:57] optimize to find the nooks and crannies

[27:59] in the model, whether it's a bigger

[28:01] model model or smaller model to improve

[28:05] the performance against your data set.

[28:07] So that's what the optimizer is doing is

[28:09] finding finding these nooks and crannies

[28:11] in the model to optimize and improve

[28:13] that performance.

[28:15] So a typical flow, I'm not going to

[28:17] spend too much time on this, but fairly

[28:19] logical constructor program which is

[28:22] decomposing your logic into the modules.

[28:24] You use your metrics to define basically

[28:26] the contours of how the program works

[28:28] and you optimize all that through um to

[28:31] to get your your uh your final result.

[28:37] So, another talk that this guy Chris

[28:39] Pototts just had maybe two days ago, um,

[28:42] where he made the point, this is what I

[28:44] was mentioning before, where Jeepa,

[28:46] which is, uh, you probably saw some of

[28:48] the the talks the other day, um, where

[28:52] the optimizers are on par or exceed the

[28:55] performance of something like GRPO,

[28:57] another kind of fine-tuning method. So,

[28:59] pretty impressive. I think it's an

[29:00] active area of research. people a lot

[29:02] smarter than me like Omar and Chris and

[29:04] others are are leading the way on this.

[29:06] But uh point being I think prompt op

[29:09] prompt optimization is a pretty exciting

[29:12] place to be and if nothing else is worth

[29:15] exploring.

[29:17] And [clears throat] then finally metrics

[29:19] again these are kind of the building

[29:21] blocks that allow you to define what

[29:23] success looks like for the optimizer. So

[29:25] this is what it's using and you can have

[29:27] many of these and we'll see examples of

[29:29] this where again at a high level your

[29:33] program works on inputs it works on

[29:35] outputs the optimizer is going to use

[29:38] the metrics to understand okay my last

[29:41] tweak in the prompt did it improve

[29:43] performance it did it degrade

[29:44] performance and the way you define your

[29:47] metrics uh provides that direct feedback

[29:49] for the optimizers to work on.

[29:53] Uh so here's another example, a super

[29:55] simple one from that time entry example

[29:57] I mentioned before. Um, so they can be

[30:01] the metrics can either be like fairly

[30:04] rigorous in terms of like does this

[30:05] equal one or or you know some type of

[30:07] equality check or a little bit more

[30:09] subjective where using LLM as a judge to

[30:11] say whatever was this generated um

[30:14] string does it adhere to these you know

[30:17] various criteria whatever it might be

[30:19] but that itself can be a metric

[30:22] and so all of this is to say it's a very

[30:24] long-winded way of saying in my opinion

[30:26] this is probably most if not all of what

[30:28] you need to construct arbitrarily

[30:31] complex workflows, data processing

[30:33] pipelines, business logic, whatever that

[30:36] might be. Different ways to work with

[30:38] LLMs. If nothing else, DSPI gives you

[30:41] the primitives that you need in order to

[30:44] build these modular composable systems.

[30:48] So, if you're interested in some people

[30:49] online, um

[30:52] there's many many more. There's a

[30:54] Discord community as well. Um, but

[30:56] usually these people are are on top of

[30:58] the latest and greatest and so would

[31:00] recommend giving them a follow. You

[31:02] don't need to follow me. I don't really

[31:03] do much, but uh the others on there are

[31:05] are really pretty good.

[31:08] Okay, so the fun part, we'll actually

[31:10] get into some to some code. So, if you

[31:13] haven't had a chance, now's your last

[31:15] chance to get the repo.

[31:17] U, but I'll just kind of go through a

[31:20] few different examples here of what we

[31:23] talked about. Maybe

[31:27] Yeah. Okay.

[31:29] Okay. So, I'll set up Phoenix, which is

[31:33] from Arise, uh, which is basically an

[31:36] obser an observability platform. Uh, I

[31:38] just did this today, so I don't know if

[31:40] it's going to work or not, but we'll

[31:41] we'll see. We'll give it a shot. Uh, but

[31:44] basically what this allows you to do is

[31:45] have a bunch of observability and

[31:47] tracing for all the calls that are

[31:48] happening under the hood. We'll see if

[31:51] this works. We'll give it like another 5

[31:52] seconds.

[31:55] Um, but it should, I think,

[31:57] automatically do all this stuff for me.

[32:00] Yeah. So, let's see.

[32:02] Yeah. All right. So, something's up.

[32:04] Okay, cool. So,

[32:07] I'll just I'm just going to run through

[32:08] the notebook, which is a collection of

[32:10] different use cases, basically putting

[32:12] into practice a lot of what we just saw.

[32:14] Feel free to jump in any questions,

[32:16] anything like that. We'll start with

[32:17] this notebook. There's a couple of other

[32:20] uh more proper Python programs that

[32:22] we'll walk through afterwards. Uh but

[32:24] really the intent is a rapidfire review

[32:26] of different ways that DSPI has been

[32:29] useful to me and others. So

[32:32] load in the end file. Usually I'll have

[32:34] some type of config object like this

[32:37] where I can very easily use these later

[32:40] on. So if I'm like call like model

[32:42] mixing. So if I have like a super hairy

[32:44] problem or like some workload I know

[32:46] will need the power of a reasoning model

[32:49] like GPD5 or something else like that,

[32:51] I'll define multiple LM. So like one

[32:54] will be 41, one will be five, maybe I'll

[32:56] do a 41 nano um you know Gemini 2.5

[33:00] flash, stuff like that. And then I can

[33:02] kind of intermingle or intersperse them

[33:04] depending on what I think or what I'm

[33:08] reasonably sure the workload will be.

[33:09] and you'll see how that comes into play

[33:11] in terms of classification and others.

[33:15] Um, I'll pull in a few others here. I'm

[33:17] I'm using open router for this. So, if

[33:20] you have an open router API key, would

[33:22] recommend plug plugging that in. So, now

[33:24] I have three different LLMs I can work

[33:26] with. I have Claude, I have Gemini, I

[33:28] have 41 mini. And then I'll ask

[33:32] basically for each of them who's best

[33:35] between Google Anthropic OpenAI. All of

[33:37] them are hedging a little bit. They say

[33:39] subjective, subjective, undefined. All

[33:41] right, great. It's not very helpful. But

[33:44] because DSPI works on Pyantic, I can

[33:48] define the answer as a literal. So I'm

[33:50] basically forcing it to only give me

[33:51] those three options and then I can go

[33:53] through each of those. And you can see

[33:55] each of them, of course, chooses their

[33:57] own organization. Um, the reason that

[33:59] those came back so fast

[34:01] is that DSP has caching automated under

[34:04] the hood. So as long as nothing has

[34:07] changed in terms of your uh your

[34:10] signature definitions or basically if

[34:12] nothing has changed this is super useful

[34:14] for testing it will just load it from

[34:16] the cache. Um so I ran this before

[34:19] that's why those came back so quickly. U

[34:22] but that's another kind of super useful

[34:25] um piece here. Let's see.

[34:32] Okay.

[34:34] Make sure we're up and running. So, if I

[34:36] change this to hello

[34:39] with a space,

[34:41] you can see we're making a live call.

[34:43] Okay, great. We're still up. So, super

[34:45] simple class sentiment classifier.

[34:47] Obviously, this can be built into

[34:48] something arbitrarily complex. Make this

[34:51] a little bit bigger. Um, but I'm

[34:53] basically I'm giving it the text, the

[34:55] sentiment that you saw before, and I'm

[34:58] adding that additional specification to

[35:00] say, okay, lower uh is more negative,

[35:02] higher is more positive. I'm going to

[35:04] define that as my signature. I'm going

[35:07] to pass this into just a super simple

[35:09] predict object.

[35:11] And then I'm going to say, okay, well,

[35:12] this hotel stinks. Okay, it's probably

[35:14] pretty negative. Now, if I flip that to

[35:17] I'm feeling pretty happy. Whoops.

[35:23] Good thing I'm not in a hotel right now.

[35:26] U you can see I'm feeling pretty happy.

[35:28] Comes down to eight. And this might not

[35:31] seem that impressive and you know it's

[35:33] it's not really but uh the the the

[35:36] important part here is that it just

[35:38] demonstrates the use of the shorthand um

[35:43] signature. So I have I have the string,

[35:44] I have the integer, I pass in the custom

[35:46] instructions which would be in the dock

[35:48] string if I use the class B classbased

[35:51] uh method. The other interesting part or

[35:54] or useful part about DSPI comes with a

[35:56] bunch of usage information built in. So

[35:59] um because it's cached, it's going to be

[36:01] an empty object.

[36:03] But when I change it, you can see that

[36:05] I'm using Azure right now, but for each

[36:08] call, you get this nice breakdown. and I

[36:09] think it's from late LLM, but allows you

[36:11] to very easily track your usage, token

[36:14] usage, etc. for observability and

[36:16] optimization and everything like that.

[36:18] Just nice little tidbits uh that are

[36:20] part of it here and there. Make this

[36:22] smaller.

[36:24] Uh we saw the example before in the

[36:26] slides, but I'm going to pull in that

[36:28] form 4

[36:30] off of online. I'm going to create this

[36:33] doc objects using attachments. You can

[36:35] see some of the stuff it did under the

[36:37] hood. So, it pulled out um PDF plumber.

[36:40] It created markdown from it. Pulled out

[36:41] the images, etc. Again, I don't have to

[36:44] worry about all that. Attachments make

[36:45] that super easy. I'm going to show you

[36:48] what we're working with here. This case,

[36:49] we have the form four. And then I'm

[36:52] going to do that poor man's rag that I

[36:54] mentioned before. Okay, great. How many

[36:56] shares were were sold in total? It's

[36:58] going to go through that whole chain of

[36:59] thought and bring back the response.

[37:01] That's all well and good, but the power

[37:04] in my mind of DSPI is that you can have

[37:08] these arbitrarily complex data

[37:10] structures. That's fairly obvious

[37:12] because it uses paidantic and everything

[37:14] else, but you can get a little creative

[37:16] with it. So in this case, I'm going to

[37:18] say, okay, a different type of document

[37:20] analyzer signature. I'm just going to

[37:21] give it the document and then I'm just

[37:23] going to defer to the model on defining

[37:26] the structure of what it thinks is most

[37:27] important from the document. So in this

[37:29] case, [clears throat] I'm defining a

[37:30] dictionary object and so it will

[37:32] hopefully return to me a series of key

[37:35] value pairs that describe important

[37:37] information in the document in a

[37:39] structured way. And so you can see here

[37:42] again this is probably cached uh but I

[37:44] passed in I did it all in one line in

[37:47] this case but I'm saying I want to do

[37:49] chain of thought using the document

[37:51] analyzer signature and I'm going to pass

[37:54] in the input field which is just the

[37:57] document here. I'm going to pass in the

[37:58] document that I got before. And you can

[38:01] see here it pulled out bunch of great

[38:03] information in the super structured way.

[38:06] And I didn't have to really think about

[38:07] it. I just kind of deferred all this to

[38:09] the model to DSPI for how to do this.

[38:12] Now, of course, you can do the inverse

[38:14] in saying, okay, I have a very specific

[38:17] business use case. I have something

[38:19] specific in terms of the formatting or

[38:22] the content that I want to get out of

[38:23] the document. I define that as just kind

[38:26] of your typical paid classes. So in this

[38:29] case I want to pull out the if there's

[38:31] multiple transactions the schema itself

[38:34] important information like the filing

[38:36] date

[38:37] going to define the document analyzer

[38:40] schema signature. Uh again super simple

[38:43] input field which is just the document

[38:45] itself which is parsed by attachments

[38:47] gives me the text and the images and

[38:50] then I'm passing in the document schema

[38:53] parameter which has the document schema

[38:55] type which is defined above and this is

[38:59] the this is effectively what you would

[39:00] pass into structured outputs um but just

[39:03] doing it the DS pie where it's going to

[39:06] give you um basically the the output in

[39:10] that specific format. So you can see

[39:12] pulled out things super nicely. Filing

[39:14] date, form date, form type, transactions

[39:17] themselves, and then the ultimate

[39:19] answer. [clears throat] And it's nice

[39:21] because it exposes it in a way that you

[39:23] can use dot notation. So you can just

[39:25] very quickly access the the resulting

[39:27] objects.

[39:29] So looking at adapters, um I'll use

[39:32] another little tidbit from DSPI, which

[39:33] is the inspect history. So for those who

[39:35] want to know what's going on under the

[39:37] hood, inspect history will give you the

[39:39] raw dump of what's actually going on. So

[39:42] you can see here the system message that

[39:43] was uh constructed under the hood was

[39:47] all of this. So you can see input fields

[39:51] are document output fields or reasoning

[39:53] and the schema. It's going to pass these

[39:56] in. And then you can see here the actual

[39:59] document content that was extracted and

[40:01] put into the text and into the prompt uh

[40:04] with some metadata. This is all

[40:05] generated by attachments. And then you

[40:07] get the response which follows this

[40:10] specific format. So you can see the

[40:11] different fields that are here. And it's

[40:13] this kind of relatively arbitrary

[40:16] response um basically format for the for

[40:20] the names which is then parsed by the

[40:21] pie and passed back to you as the user.

[40:24] Um, so I can do okay response.document

[40:27] schema and get the the actual result.

[40:31] To show you what the BAML adapter looks

[40:33] like, we can basically do two different

[40:35] calls. So this is an example from uh my

[40:38] buddy Pashant uh online again. So what

[40:42] we do here is define pyantic model super

[40:45] simple one. Patient address and then

[40:47] patient details. Patient details has the

[40:50] patient address object within it. And

[40:53] then we're going to say we're going to

[40:54] create a super simple DSPI signature to

[40:57] say taking a clinical note which is a

[40:59] string. The patient info is the output

[41:01] type. And then note so I'm going to run

[41:03] this two different ways. The first time

[41:05] with the smart LLM that I mentioned

[41:08] before and just use the the built-in

[41:11] adapter. So I don't specify anything

[41:13] there. And then the second one will be

[41:16] using the BAML adapter which which is

[41:18] defined there. Um so I guess a few

[41:21] things going on here. One is the ability

[41:23] to use Python's uh context which is the

[41:27] the lines starting with with width which

[41:30] allow you to basically break out of what

[41:32] the global LLM um has been defined as

[41:36] and use a specific one just for that

[41:37] call. So you can see in this case I'm

[41:40] using the same LM but if I want to

[41:42] change this to like LM anthropic or

[41:44] something

[41:47] I think that should work. Um, but

[41:49] basically what that's doing is just

[41:51] offloading that call to the other

[41:53] whatever LLM that you're defining

[41:55] [clears throat] for that particular call

[41:56] and something happened. And I'm on a

[41:59] VPN, so let's kill that.

[42:03] Sorry, Alex Partners.

[42:06] Okay.

[42:10] Okay, great. So, we had two separate

[42:12] calls. One was to the smart LLM, which

[42:13] is I think 41. The other one was to

[42:16] Anthropic. Same. Everything else is the

[42:18] exact same. The notes exact same, etc.

[42:20] We got the same exact output. That's

[42:22] great. But what I wanted to show here is

[42:25] the adapters themselves. So in this

[42:28] case, I'm doing inspect history equals

[42:30] 2. So I'm going to get both of the last

[42:32] two calls. And we're going to see how

[42:34] the prompts are going to be different.

[42:37] And so you can see here the first one,

[42:39] this is the built-in JSON schema, this

[42:42] crazy long JSON string. Yeah, LLMs are

[42:45] good enough to to handle that, but um

[42:48] you know, probably not for super

[42:50] complicated ones. Um uh and then you see

[42:54] here for the the second one, it uses the

[42:57] BAML notation, which as we saw in the

[42:59] slides, a little bit easier to

[43:00] comprehend. Um and on super complicated

[43:03] use cases can actually have a measurable

[43:05] u improvement.

[43:08] Multimodal example, same sort of thing

[43:10] as before. I'll pull in the image

[43:11] itself.

[43:13] Let's just see what we're working with.

[43:14] Okay, great. We're looking at these

[43:16] various street signs.

[43:18] And I'm just going to ask it super

[43:21] simple question. It's this time of day.

[43:23] Can I park here now? When when should I

[43:25] leave? And you can see I'm just passing

[43:28] in again the super simple um shorthand

[43:32] for defining a signature which then I

[43:34] get out the the var the boolean in this

[43:37] case and a string of when I can leave.

[43:40] Um

[43:41] so modules themselves it's again fairly

[43:45] simple. You just kind of wrap all this

[43:46] in a class. Good question.

[43:48] >> So does it return reasoning by default

[43:50] always?

[43:51] >> Oh good question. Yeah. So when you do

[43:53] >> can you repeat the question?

[43:55] >> Yes. So for those online the question

[43:57] was does it always return reasoning by

[43:59] default? When you call DSPI.chain chain

[44:02] of thought as part of the module where

[44:05] it's built in. It's adding the reasoning

[44:08] u automatically into your response. So

[44:10] you're not defining that. It's a great

[44:11] question. It's not defined in the

[44:13] signature as you can see up here. Uh but

[44:16] it will add that in and expose that to

[44:18] you um to the extent that you want to

[44:21] retain it for any you know any reason.

[44:23] Uh but that's so if I ju if I changed

[44:26] this to predict

[44:29] you wouldn't get that same response,

[44:32] right? You just you literally just get

[44:34] that part.

[44:36] Um so that's actually a good segue to

[44:38] the modules. Um so module is basically

[44:40] just wrapping all that into some type of

[44:43] replic replicable uh logic. Um and so

[44:47] we're just we're giving it the signature

[44:49] here. We're saying selfpredict.

[44:52] We're in this case is just a

[44:53] demonstration of how it's being used as

[44:56] a class. So I'll just add this module

[44:58] identifier and sort some sort of counter

[45:00] but this can be any type of arbitrary

[45:02] business logic or control flow or any

[45:05] database action or whatever it might be.

[45:07] When this image analyzer class is called

[45:09] this function would run um and then when

[45:12] you actually invoke it this is when it's

[45:14] actually going to run the the core

[45:16] logic. And so you can see I'm just

[45:17] passing in the So I'm instantiating it

[45:20] the analyzer of AIE123

[45:22] and then I'll call it.

[45:25] Great. It called that and you can see

[45:27] the counter incrementing each time I

[45:29] actually make the call. So super simple

[45:31] example. Um we don't have a ton of time

[45:34] but I'll I'll show you some of the other

[45:35] modules and how that kind of works out.

[45:38] Terms of tool calling fairly

[45:40] straightforward. I'm going to define two

[45:41] different functions perplexity search

[45:43] and get URL content. creating a bioagent

[45:46] module. So this is going to define

[45:50] Gemini 25 as this particular module's um

[45:53] LLM. It's going to create an answer

[45:55] generator object which is a react call.

[45:58] So I'm going to basically do tool

[46:01] calling whenever this is called and then

[46:03] the forward function is literally just

[46:05] calling that answer generator with the

[46:07] parameters that are provided to it. And

[46:09] then I'm creating an async version of

[46:11] that function as well.

[46:13] So I can do that here. I'm going to say

[46:15] okay identify instances where a

[46:18] particular person has been at their

[46:19] company for more than 10 years. It needs

[46:21] to do tool calling to do this to get the

[46:24] most up-to-date information. And so what

[46:25] this is doing and basically looping

[46:27] through um and it's going to call that

[46:29] bio agent which is using the tool calls

[46:31] in the background and it will make a

[46:34] determination as to whether their

[46:35] background is applicable per my

[46:37] criteria. In this case, Satia is true.

[46:40] Brian should be false. Uh but what's

[46:43] interesting here while that's going in

[46:45] it uh similar to the reasoning uh par or

[46:48] the reasoning object that you get back

[46:50] for chain of thought you can get a

[46:51] trajectory back for things like react.

[46:55] So you can see what tools it's calling

[46:57] the arguments that are passed in um and

[47:00] the observations for each of those calls

[47:02] which is nice for debugging and and

[47:04] other obviously other uses.

[47:07] Um I want to get to the other content so

[47:09] I'm going to speed through the rest of

[47:10] this. This is basically an async version

[47:12] of the same thing. So you would run both

[47:14] of them in parallel. Same idea.

[47:17] Um I'm going to skip the JEPA example

[47:20] here just for a second. Um I can show

[47:22] you what the output looks like, but

[47:24] basically what this is doing is creating

[47:28] a data set.

[47:30] It is showing you what's in the data

[47:32] set. It's creating a variety of

[47:34] signatures. In this case, it's going to

[47:36] create a system that categorizes and

[47:38] classifies different basically help

[47:41] messages um that is part of the data

[47:43] set. So, my sync is broken or my light

[47:46] is out or whatever it is. They want to

[47:48] classify whether it's positive, neutral,

[47:49] or negative and the uh the urgency of

[47:52] the actual message. It's going to

[47:55] categorize it and then it's going to

[47:56] pack all this stuff, all those different

[47:58] modules into a single support analyzer

[48:02] module. And then from there, what it's

[48:04] going to do is define a bunch of metrics

[48:07] which is based off of the data set

[48:09] itself. So it's going to say, okay, how

[48:11] do we score the urgency? This is a a

[48:14] very simple one where it's okay, it

[48:16] either matches or it doesn't. Um, and

[48:19] there's other ones where it can be a

[48:21] little bit more subjective and then you

[48:23] can run it. This going to take too long.

[48:25] Probably takes 20 minutes or so. Um but

[48:29] uh what it will do is basically evaluate

[48:31] the performance of the base model and

[48:33] then apply those metrics uh and

[48:36] iteratively come up with new prompts to

[48:39] uh to create that.

[48:41] Now I want to pause here just for a

[48:42] second because there's different types

[48:45] of metrics and in particular for Jeepa

[48:48] it uses feedback from the teacher model

[48:51] in this case. So it can work with the

[48:54] same level of model, but in particular

[48:55] when you're trying to use say a smaller

[48:58] model, um it can actually provide

[49:00] textual feedback. So, it says not only

[49:02] did you get this classification wrong,

[49:04] but it's going to give you some

[49:06] additional um information or feedback as

[49:10] you can see here for why it got it wrong

[49:12] or what the answer should have been,

[49:14] which allows it you you can read the

[49:16] paper, but it basically allows it to um

[49:19] iteratively find that kind of paro

[49:21] frontier of how it should uh tweak the

[49:25] prompt to optimize it based off that

[49:26] feedback. It basically just tightens

[49:28] that iteration loop.

[49:31] Um you can see there's a bunch here. Um

[49:34] and then you can run it and see how it

[49:36] works. [snorts] Um but kind of just to

[49:39] give you a concrete example of how it

[49:40] all comes together. So we took a bunch

[49:44] of those examples from before. We're

[49:46] basically basically going to do a bit of

[49:50] um categorization. So I have things like

[49:54] contracts, I have images, I have

[49:57] different things that one DSPI program

[50:01] can comprehend and do some type of

[50:03] processing with. So this is something

[50:04] that we see fairly regularly in terms of

[50:07] we might run into a client situation

[50:09] where they have just a big dump of of

[50:12] files. They don't really know what's in

[50:14] it. They want to find something of uh

[50:16] they want to maybe find SEC filings and

[50:19] process them a certain way. they want to

[50:21] find contracts and process those a

[50:23] certain way. Maybe there's some images

[50:25] in in there and they want to process

[50:26] those a certain way. Uh [snorts] so this

[50:28] is an example of how you would do that

[50:30] where if I start at the bottom here,

[50:34] this is a regular Python file. Um and it

[50:38] uses DSPI to do all those things I just

[50:40] mentioned. So we're pulling in the

[50:42] configurations,

[50:44] we're setting the regular LM, the small

[50:46] and one we use for an image. As an

[50:48] example, Gemini might Gemini models

[50:51] might be better at image recognition

[50:53] than others. So I might want to defer or

[50:55] use a particular model for a particular

[50:58] workload. So if I detect an image, I

[51:01] will route the request to Gemini. If I

[51:03] detect something else, I'll route it to

[51:05] a 4.1 or whatever it might be.

[51:09] So I'm going to process a single file.

[51:13] And what it does is use our handy

[51:17] attachments

[51:18] um library to put it into a format that

[51:21] we can use. And then I'm going to

[51:24] classify it. And it's not super obvious

[51:27] here, but I'm getting a file type from

[51:29] this classify file uh function call. And

[51:32] then I'm doing some different type of

[51:34] logic depending on what type of file it

[51:36] is. So if it's an SEC filing, I do

[51:39] certain things. If it's a certain type

[51:42] of SEC filing, I do something else. Uh,

[51:44] if [snorts] it's a contract, maybe I'll

[51:46] summarize it. If it's something that

[51:48] looks like city infrastructure, in this

[51:49] case, the image that we saw before, I

[51:51] might do some more visual interpretation

[51:53] of it. Um, so if I dive into classify

[51:57] file super quick,

[52:00] it's running the document classifier.

[52:03] And all that is is basically doing a

[52:06] predict on the image from the file. and

[52:12] um making sure

[52:15] it returns a type. Where is this

[52:19] returns a type which would be document

[52:22] type and so you can see here at the end

[52:24] of the day it's a fairly simple

[52:26] signature and so what we've done is

[52:28] basically take the PDF file in this case

[52:31] take all the images from it and take the

[52:34] first image or first few images in this

[52:36] case a list of images as the input field

[52:39] and I'm saying okay just give me the

[52:41] type what is this and I'm giving it an

[52:43] option of these document types so

[52:46] obviously say this is a fairly simple

[52:48] use case but it's basically saying given

[52:51] these three images the first three pages

[52:53] of a document is it an SEC filing is it

[52:55] a patent filing is the contract city

[52:57] infrastructure pretty different things

[53:00] so the model really shouldn't have an

[53:01] issue with any of those and then we have

[53:02] a catchall bucket for other and then as

[53:05] I mentioned before um depending on the

[53:08] file type that you get back you can

[53:10] process them differently so I'm using

[53:12] the small model to do the same type of

[53:15] form4 extraction that we saw before um

[53:20] and then asserting basically in this

[53:22] case that it is what we think it is. Um

[53:24] a contract in this case we're saying uh

[53:28] let's see I have like 10 more minutes so

[53:31] we can go we'll we'll stop after this uh

[53:33] up to this file but for the particular

[53:37] contract we'll go we'll create this

[53:38] summarizer object. So we'll go through

[53:41] as many pages as there are. We'll do

[53:43] some uh basically recursive

[53:45] summarization of that using a separate

[53:47] DSPI function and then we'll detect some

[53:50] type of boundaries of that document too.

[53:53] So we'll say I want the summaries and I

[53:55] want the boundaries of the document. Um

[53:57] and then we'll print those things out.

[53:59] So let's just see if I can run this.

[54:02] It's going to classify it should as a

[54:06] [clears throat] contract.

[54:12] >> So is you're just relying on the model

[54:14] itself to realize that it's a city

[54:16] infrastructure.

[54:18] >> Yeah. The question was I'm I'm just

[54:20] relying on the model to determine if

[54:22] it's a city infrastructure. Yes. I mean

[54:25] this is more just like a workshop quick

[54:27] and dirty example. It's only because

[54:29] there's one picture of the street signs.

[54:31] Um, and if we look in the data folder, I

[54:35] have a contract,

[54:36] some image that's irrelevant, the form

[54:39] for SEC filing, and then the parking

[54:41] too. Um, they're pretty different. The

[54:44] model should have no problem out of

[54:45] those categories that I gave it to

[54:47] categorize it properly. In some type of

[54:49] production use case, you would want much

[54:51] more stringent or maybe even multiple

[54:54] passes of classification, maybe using

[54:56] different models to do that. Um but

[54:59] yeah, given those options, at least the

[55:01] many times I've run it, had no problem.

[55:04] So in this case, I gave it um one of

[55:08] these contract documents and it ran some

[55:11] additional summarization logic under the

[55:13] hood. So, if I go to that super quick,

[55:16] um you can find all this in the code,

[55:17] but basically what it does is use three

[55:20] separate signatures to basically

[55:23] decompose the contents of the the um the

[55:27] contract and then summarize them up. So,

[55:29] it's basically just iteratively working

[55:31] through each of the chunks of the

[55:33] document to create a summary that you

[55:36] see here at the bottom. And then just

[55:38] for good measure, we're also detecting

[55:40] basically the the boundaries of the

[55:42] document to say, okay, here's out of the

[55:45] 13 pages, you have the main document and

[55:48] then some of the exhibits or the

[55:49] schedules that are a part of it. So, let

[55:52] me just bring it up super quick

[55:57] just to show you what we're working

[55:59] with. This is just some random thing I

[56:01] found online. And you can see so it said

[56:06] the main document was from page 0 to six

[56:11] and the way and so we zero 1 2 3 4 5 six

[56:16] seems reasonable. Now we have the start

[56:18] of schedule one.

[56:21] Schedule one it says it's the next two

[56:23] pages. That looks pretty good. Schedule

[56:26] two is just the one page 9 to9.

[56:31] That looks good. and then schedule three

[56:33] through to the end of the document.

[56:36] And that looks pretty good, too. And so

[56:38] the way we did that under the hood was

[56:40] basically take the PDF, convert it to a

[56:43] list of images and then for each of the

[56:45] images pass those to classifier

[56:48] um and then use that to

[56:52] well let's just look at the code but

[56:53] basically take the list of those

[56:55] classifications

[56:57] give that to another DSPI signature to

[56:59] say given these classifications of the

[57:01] document give me the structure and

[57:04] basically give me a key pair of you name

[57:08] of the section and two integers, a

[57:10] tupole of integers that detect or that

[57:13] uh determine the um you know the

[57:15] boundaries essentially. Um so that's

[57:18] what that part does.

[57:20] Um [clears throat]

[57:22] if we go back so city infrastructure,

[57:26] I'll do this one super quick just

[57:27] because it's pretty interesting on how

[57:28] it uses tool calls. And while this is

[57:31] running,

[57:33] I should use the right one. Hold on.

[57:37] >> [clears throat]

[57:40] >> Yeah,

[57:40] >> good question. The second part like when

[57:43] you generated the list of like my

[57:44] documents from 0 to six, did you have

[57:46] like original document as an input or

[57:48] no?

[57:49] >> No. Uh so let let's just go to that uh

[57:52] that was super quick. So

[57:55] that should be boundary detector.

[58:00] So, there's a blog post on this that I

[58:02] published probably in August or so that

[58:04] goes into a little bit more detail. The

[58:05] code is actually pretty crappy in that

[58:07] one. It's it's going to be better here.

[58:08] Um, but basically what it does is

[58:14] this is probably the main logic. So, for

[58:16] each of the images in the PDF, we're

[58:20] going to call classify page.

[58:23] We're going to gather the results. So

[58:25] it's doing all that asynchronously

[58:27] pulling it all back saying okay all

[58:29] these you know all the different page

[58:30] classifications that there are and then

[58:32] I pass the output of that into a new

[58:35] signature that says given tupil of p I

[58:39] don't even define it here given tupil of

[58:41] page and classification

[58:44] give me this I don't know relatively

[58:46] complicated output of a dictionary of a

[58:49] string tupil integer integer and I give

[58:53] it this set of instructions to say just

[58:56] detect the boundaries. Like this is all

[58:58] very like non-production code obviously,

[59:01] but the point is that you can do these

[59:03] types of things super super quickly.

[59:05] Like I'm not specifying much not giving

[59:07] it much context and it worked like

[59:09] pretty well. Like it it's worked pretty

[59:12] well in most of my testing. Now

[59:14] obviously there is a ton of low hanging

[59:15] fruit in terms of ways to improve that,

[59:17] optimize it, etc.

[59:19] Um, but all this is doing is taking that

[59:23] signature, these instructions, and then

[59:26] I call react. And then all I give it is,

[59:30] uh, the ability to basically

[59:32] self-reflect and call um, get page

[59:35] images. So, it says, okay, I'm going to

[59:36] look at this boundary. Well, let me get

[59:38] the the page images for these three

[59:40] pages to and make sure basically that

[59:43] the boundary is correct. And then it

[59:45] uses that to construct the final answer.

[59:48] And so it's really this is a perfect

[59:50] example of like the tight iteration loop

[59:52] that you can have both in um building it

[59:55] but then the you can kind of take

[59:57] advantage of the model's introspective

[59:59] ability if you will to use function

[60:01] calls against the data itself the data

[60:04] it generated itself etc to kind of keep

[60:07] that loop going. question.

[60:10] >> So under the hood, the the beauty of ESP

[60:14] then is that it enforces kind of

[60:16] structured output on a on a model.

[60:20] >> I mean yes, I think that's probably

[60:23] reductive of of like its full potential,

[60:25] but generally that's that's correct. I

[60:27] mean yes, you can use structured

[60:28] outputs, but you have to do a bunch of

[60:31] crap basically to coordinate like

[60:34] feeding all the feeding that into the

[60:36] rest of the program. maybe you want to

[60:38] call a model differently or use XML here

[60:40] or use a different type of model or

[60:42] whatever it might be

[60:45] um to to do that. So absolutely yeah I'm

[60:48] not saying this is the only way

[60:49] obviously to kind of create these

[60:50] applications or that you shouldn't use

[60:52] Pantic or shouldn't use structured

[60:54] outputs. You absolutely should. Um, it's

[60:56] just a way that once you kind of wrap

[60:59] your head around the the primitives that

[61:00] DSPI gives you, you can start to very

[61:04] quickly build these types of

[61:06] arguably uh I mean these are like

[61:09] prototypes right now, but like if you

[61:11] want to take this to the next level to

[61:13] production scale, you have all the

[61:14] pieces in front of you to be able to do

[61:15] that.

[61:17] >> Um,

[61:19] any other questions? I probably got

[61:21] about five minutes left. Go ahead. Can

[61:22] you talk about your experience using

[61:25] optimization

[61:26] and just

[61:29] >> Yeah. Yeah. So Jeep and actually I'll

[61:31] pull up uh I I ran one right before

[61:34] this. Um this uses a a different

[61:37] algorithm called my row but basically um

[61:41] the optimizers as long as you have well

[61:44] structured data. So for the machine

[61:46] learning folks in the room, which is

[61:47] probably everybody, obviously the

[61:50] quality of your of your data is very

[61:51] important,

[61:53] um you don't need thousands and

[61:54] thousands of examples necessarily, but

[61:56] as long as you have enough, maybe 10 to

[61:59] 100 of inputs and outputs.

[62:01] [clears throat] And if you're

[62:03] constructing your metrics in a way that

[62:04] is relatively intuitive and and that,

[62:07] you know, accurately describes what

[62:09] you're trying to achieve, the

[62:11] improvement can be pretty significant.

[62:14] Um, and so that time entry corrector

[62:16] thing that I mentioned before, uh, you

[62:18] can see the output of here. It's kind of

[62:20] iterating through. It's measuring the

[62:22] output metrics for each of these. And

[62:25] then you can see all the way at the

[62:26] bottom once it goes through all of its

[62:28] optimization stuff.

[62:31] You can see the actual performance

[62:34] on

[62:36] um, the basic versus the optimized

[62:39] model. In this case, went from 86 to 89.

[62:42] And then interestingly, this is still in

[62:44] development, this one in particular, but

[62:46] you can break it down by metric. So you

[62:47] can see where the model's optimizing

[62:49] better, performing better across certain

[62:52] metrics. And this can be really telling

[62:54] as to whether you need to tweak your

[62:56] metric, maybe you need to decompose your

[62:58] metric, maybe there's other areas within

[63:01] your data set, or the the basically the

[63:04] structure of your program that you can

[63:06] improve. Um, but it's a really nice way

[63:08] to understand what's going under the

[63:10] under the hood. And if if you don't care

[63:13] about some of these and the optimizer

[63:15] isn't doing as well on them, maybe you

[63:17] can maybe you can throw them out, too.

[63:18] So, it's it's a very kind of flexible

[63:21] system, flexible way of kind of doing

[63:22] all that.

[63:23] >> Yeah. What's the output of the

[63:25] optimization? Like what do you get out

[63:26] of it and then how do you use that

[63:28] object, whatever it is?

[63:29] >> Yeah. Yeah. So the output of the

[63:30] optimizers is basically just another um

[63:34] it's almost like a compiled object if

[63:36] you will.

[63:37] >> So DSPI allows you to save and load

[63:39] programs as well. So the output of the

[63:41] optimizer is basically just a module

[63:44] that you can then serialize and save off

[63:46] somewhere

[63:47] >> or you can call it later uh as you would

[63:49] any other module

[63:51] >> and it's just manipulating the phrasing

[63:53] of the prompt. So like what is it

[63:55] actually like you know what's its

[63:56] solution space look like?

[63:58] >> Yeah. Yeah. under the hood, it's

[63:59] literally just iterating on the actual

[64:01] prompt itself. Maybe it's adding

[64:03] additional instructions. It's saying,

[64:05] "Well, I keep failing on this particular

[64:07] thing, like not capitalizing the names

[64:09] correctly. I need to add [clears throat]

[64:10] in my upfront criteria in the prompt an

[64:13] instruction to the model to say you must

[64:15] capitalize names properly." And Chris uh

[64:18] who I mentioned before has a really good

[64:20] way of putting this and I'm going to

[64:21] butcher it now, but like the optimizer

[64:23] is basically finding latent requirements

[64:25] that you might not have specified

[64:27] initially up front, but based off of the

[64:29] data, it's kind of like a poor man's

[64:31] deep learning, I guess, but like it's

[64:33] learning from the data. It's learning

[64:34] what it's doing well, what what it's

[64:35] doing not so well, and it's dynamically

[64:38] constructing a prompt that improves the

[64:40] performance based off of your metrics.

[64:42] And is that like LMG guided like is it

[64:44] like about like capitalization?

[64:47] >> Yeah. Yeah. Question being is it all LLM

[64:48] guided? Yes. Particularly for Jeepa it's

[64:51] using LLM to improve LLM's performance.

[64:54] So it's using the LLM to dynamically

[64:56] construct new prompts which are then fed

[64:59] into the system measured and then it

[65:02] kind of iterates. So it's using AI to

[65:04] build AI if you will.

[65:06] >> Thank you.

[65:07] >> Yeah

[65:09] question. Why is the solution object not

[65:11] just the optimized prompt?

[65:12] >> Why is the solution object not what?

[65:14] >> Not just the optimized prompt. Why are

[65:16] you using

[65:17] >> Oh, absolutely is. You can get it under

[65:19] the hood. I mean, you can The question

[65:22] was why don't you just get the optimized

[65:24] prompt? You can absolutely. Um,

[65:26] >> what else is there besides

[65:30] >> the the So, what else is there other

[65:32] than the prompt? The DSPI object itself.

[65:35] So the module the way things um well we

[65:39] can probably look at one if we have

[65:41] time. Um

[65:43] >> if I could see a dump of what gets you

[65:45] know what is the optimized state that

[65:46] would be interesting.

[65:47] >> Yeah. Yeah sure. Let me see if I can

[65:48] find one quick. Um but fundamentally at

[65:52] the end of the day yes you get an

[65:53] optimized prompt a string that you can

[65:55] dump somewhere if you if you want to. Um

[65:58] actually

[66:00] um

[66:03] >> there's a lot of pieces to the

[66:04] signature, right? So it's like how you

[66:06] describe your feels in the doc.

[66:08] >> This is a perfect segue and I'll I'll

[66:10] conclude right after this. I was playing

[66:12] around with something I was well I was

[66:14] playing around this thing called DSPIHub

[66:17] that I kind of created to create a

[66:20] repository of optimized programs. So

[66:22] basically like if you're an expert in

[66:24] whatever you optimize an LLM against

[66:27] this data set or have a great classifier

[66:30] for city infrastructure images or

[66:32] whatever kind of like a hugging face you

[66:35] can download something that has been

[66:37] pre-optimized

[66:39] and then what I have here this is the

[66:42] actual loaded program this would be the

[66:44] output of the optimized process or it it

[66:47] is and then I can call it as I would any

[66:50] anything else and so you You can see

[66:52] here this is the output and I used the

[66:54] optimized program that I downloaded from

[66:57] from this hub. And if we inspect maybe

[67:00] the loaded program,

[67:04] you can see under the hood, it's a

[67:06] predict object with a string signature

[67:09] of time and reasoning. Here is the

[67:12] optimized prompt. Ultimately,

[67:16] this is the output of the optimization

[67:18] process. this long string here.

[67:21] Um, and then the various uh

[67:24] specifications and definitions of the

[67:26] inputs and outputs.

[67:27] >> Have you found specific uses of those?

[67:29] Like to his question like what is it?

[67:31] What can you do with that?

[67:33] >> It's up to your it's up to your use

[67:35] case. So if I if I have a so a document

[67:38] classifier might be a good example. If

[67:41] in my business I come across whatever

[67:43] documents of a certain type, I might

[67:45] optimize a classifier against those and

[67:48] then I can use that somewhere else on a

[67:51] different project or something like

[67:52] that. So out of 100,000 documents, I

[67:56] want to find only the pages that have an

[67:59] invoice on it as an example. Now sure

[68:02] 100% you can use a typical ML classifier

[68:04] to do that. That's great. This is just

[68:07] an example. We can also theoretically

[68:10] train or optimize a model to do that

[68:13] type of classification or some type of

[68:15] generation of text or what have you

[68:17] which then you have the optimized state

[68:19] of which then lives in your data

[68:22] processing pipeline you know and you can

[68:25] use it for other types of purposes or

[68:27] give it to other teams or whatever it

[68:29] might be. So it's just up to your

[68:31] particular use case. um something like

[68:34] this like hub who maybe it's not useful

[68:37] because each individual's use case is so

[68:40] hyper specific I don't really know but

[68:42] um yeah you can do with it kind of

[68:45] whatever you want last question yeah

[68:49] >> is generally you know like using DSP

[68:53] something where people kind of do

[68:54] replays just to optimize their prop or

[68:57] is there a way to sort of do it in real

[68:59] time given delays

[69:02] What I mean by delayed is okay chat GPT

[69:05] gives you your answer and you can thumbs

[69:08] up or thumbs down. You know that thumbs

[69:10] up comes you know 10 minutes later, 30

[69:13] minutes later, a day later, right?

[69:15] >> So is the question more about like

[69:17] continuous learning like how would you

[69:19] do that here?

[69:22] >> You can be the judge.

[69:26] >> Well, how are you feeding back delayed

[69:28] metrics to optimize it? Why why would it

[69:32] need to be delayed? Because you know

[69:35] usually the feedback is from the user,

[69:37] right? Like delayed.

[69:42] >> Yeah. Well, then

[69:43] >> yeah, that's right. You it basically be

[69:45] added to the data set and then you would

[69:48] use the latest optimize and just keep

[69:50] keep optimizing off of that

[69:51] >> ground truth data set.

[69:52] >> That's right.

[69:53] >> You will collect the outputs of your

[69:55] optimization and feed it back and the

[69:58] loop hits.

[70:00] >> Yeah. But that Why you're trying to do

[70:01] offline optimization, right?

[70:04] >> Yes.

[70:04] >> But I'm I'm asking, can you do this

[70:07] online where with the metric feedback?

[70:11] >> If you're good if you're a good enough

[70:12] engineer, you probably do it. But

[70:14] >> I'm not I'm not recommending replacing

[70:16] ML models with like optimized DSPI

[70:19] programs for particular use cases. Maybe

[70:21] like classification is a terrible

[70:23] example. I recognize that. But for other

[70:25] other are other in theory, yes, you

[70:28] know, you could do something like that.

[70:29] Yes.

[70:32] But for for particular LLM tasks, I'm

[70:34] sure we all have interesting ones. If

[70:37] you have something that is relatively

[70:38] well defined where you have known inputs

[70:41] and outputs, it might be a candidate for

[70:44] something worth optimizing. If nothing

[70:46] else, to transfer it to a smaller model

[70:49] to preserve the level of performance at

[70:50] a lower cost. That's really one of the

[70:52] biggest benefits I see.

[70:56] All right, last last question.

[70:59] I've heard that uh DSPI is can be kind

[71:01] of expensive because you're doing all

[71:03] these LM calls.

[71:05] >> Um so I was curious your experience with

[71:07] that and maybe relatedly like if you

[71:09] have any experience with like large

[71:12] context in your optimization data set

[71:16] ways of shrinking those.

[71:18] >> Yeah. So the question was do can BSI be

[71:20] expensive and then for large context

[71:23] kind of how have you seen that? How have

[71:24] you managed that? The expensive part is

[71:26] totally up to you. If you call a

[71:29] function a million times asynchronously,

[71:32] you're going to generate a lot of cost.

[71:33] I don't think DSPI necessarily maybe it

[71:35] makes it easier to call things, but it's

[71:38] not inherently expensive. It might, to

[71:40] your point,

[71:44] add more content to the prompt. Like,

[71:46] sure, the signature is a string, but the

[71:48] actual text that's sent to the model is

[71:50] much longer than that. That's totally

[71:52] true. I wouldn't say that it's a large

[71:55] cost driver. I mean it again it's

[71:57] ultimately it's more more of a

[71:58] programming paradigm. So you can write

[72:01] your compressed adapter if you want that

[72:03] like you know reduces the amount that's

[72:05] sent to the to uh to the model. Um in

[72:08] terms of large context I it's kind of

[72:10] the same answer I think in terms of if

[72:12] you're worried about that maybe you have

[72:14] some additional logic either in the

[72:16] program itself or in an adapter or part

[72:19] of the module that keeps track of that.

[72:21] Maybe you do some like context

[72:22] compression or something like that.

[72:24] There's some really good talks about

[72:25] that past few days. Obviously, I have a

[72:28] feeling that that will kind of go away

[72:30] at some point where either context

[72:33] windows get bigger or context management

[72:36] is abstracted away somehow. I don't

[72:38] really have an answer just that's more

[72:39] of an intuition. Um, but DSP again kind

[72:42] of gives you the tools, the primitives

[72:44] for you to do that should you choose.

[72:46] Um, and kind of track that state, track

[72:48] that management over time.

[72:50] So, I think that's it. We're going to

[72:51] get kicked out soon. So, thanks so much

[72:53] for your time. Really appreciate it.

[72:56] [music]

[73:06] [music]